Notification Systems

the right message, once, on the right channel

Watch the 4-min video

In one line: services publish **events**; one notification service resolves recipients, checks **preferences**, renders **templates**, de-duplicates with an **idempotency key** per event-user-channel, and hands the result to **priority queues** whose workers call APNs, FCM, SMS and email providers with retries, failover and per-user rate limits.

1 Twice, at 3 a.m.

Your order ships and your phone buzzes twice with the same message. Then an email arrives at 3 a.m., although you turned email off. Each bug has a different cause: a retried event (duplicate), a missing preference check (email), no quiet hours (3 a.m.). Sending a notification is one API call; sending the *right* one, *once*, on the right channel at the right time is a system.

2 Channels and their rules

Channel	Provider / protocol	Limits and quirks
iOS push	APNs over HTTP/2, token (JWT) or certificate auth	payload up to 4 KB; `apns-priority` 10 (now) or 5 (power-saving); `apns-collapse-id` replaces an earlier alert; HTTP 410 = token no longer valid
Android / web push	FCM HTTP v1 API, OAuth 2 service account	4 KB payload; high vs normal priority; `collapse_key`; `UNREGISTERED` error = delete the token
SMS	aggregators (Twilio, Vonage ...)	160 GSM-7 characters per segment (70 with Unicode); real cost per segment; country rules and sender registration
Email	SMTP or provider API (SES, SendGrid ...)	SPF, DKIM and DMARC for deliverability; handle bounces and complaints; one-click unsubscribe header (RFC 8058) for bulk mail
In-app	your own inbox table + WebSocket (lesson 26)	fully under your control; needs read/unread state

Because each channel differs, producers must never call providers directly. The order service publishes `order_shipped`; the notification service decides who gets what, where. Adding WhatsApp later is one new channel adapter, not a change in fifty services.

3 The pipeline

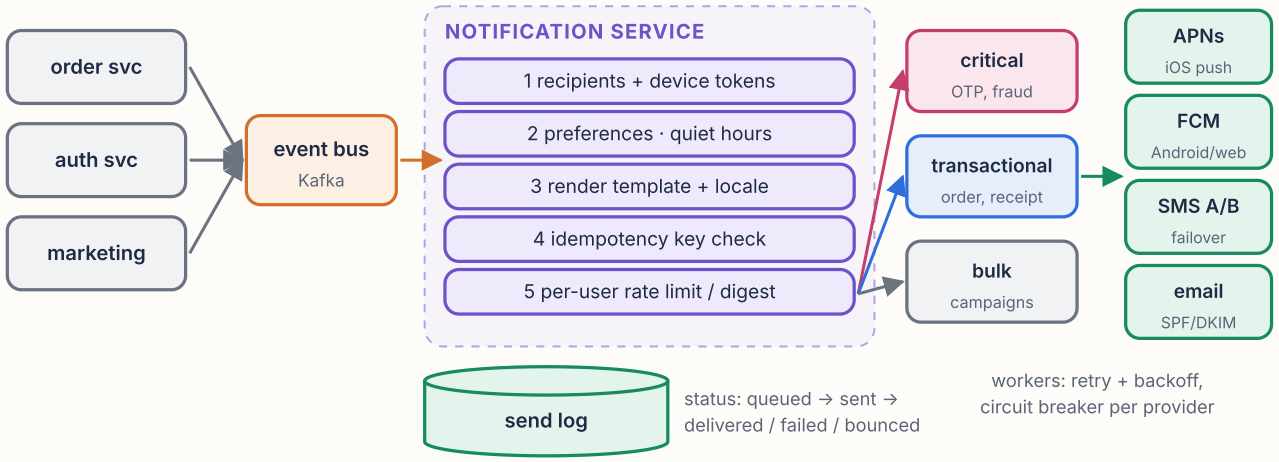


Figure 1. Events come in once; the notification service applies the same five checks to every message, then routes by priority to channel workers that talk to the providers.

Recipients and devices. A device-token registry maps users to their push tokens (one per app install). Tokens go stale when apps are uninstalled; delete them when APNs answers 410 or FCM answers `UNREGISTERED`, or you keep paying for pushes that go nowhere.

Preferences first. Before rendering anything, check: is this category on (security, orders, marketing)? Is this channel on and verified (confirmed email, phone)? Is it quiet hours in *the user's* time zone? Marketing needs explicit consent; security messages such as password resets and fraud alerts usually bypass quiet hours and cannot be turned off. Preferences live in their own store, cached, with safe defaults when the lookup fails (for marketing: don't send).

Templates. The event carries data (`order_id`, `delivery_date`); a template per channel and locale turns it into a 40-character push title, a full HTML email or a 160-character SMS. Templates are versioned outside the code, so copy changes need no deploy, and fall back by locale (`pt-BR → pt → en`). Log the template version with every send.

4 Delivery: priorities, retries, failover, limits

- **Priority queues.** Separate queues and worker pools per priority, so a 10-million-user campaign cannot delay a one-time password. Bulk sends are also throttled to what providers and your own backend (deep links!) can absorb.
- **Retries.** Retry transient errors (5xx, timeouts, 429) with exponential backoff and jitter, honouring `Retry-After`. Never retry permanent errors (invalid token, hard bounce, unsubscribed number); after N attempts, move to a dead-letter queue.
- **Failover.** For SMS and email, keep a second provider behind a circuit breaker: when error rates spike, switch. Push has no alternative provider, so the fallback is another channel (email, SMS) for critical messages.
- **Per-user rate limits and digests.** Cap each user per channel (e.g. a few pushes per hour). Past the cap, collapse into a digest ("5 people liked your photo") instead of fifty separate buzzes.
- **Status tracking.** Record every attempt: queued, sent, delivered (from provider callbacks/webhooks), opened, failed, bounced. Support can then answer "did the user get it?" in seconds.

5 No duplicates: idempotency keys

Event buses deliver **at least once**, and a worker can crash after calling the provider but before recording it. So derive a deterministic key from `(event_id, user_id, channel)` and claim it atomically before sending, e.g. Redis `SET key 'sending' NX EX 604800` or a unique index in the send log. If the key exists, skip.

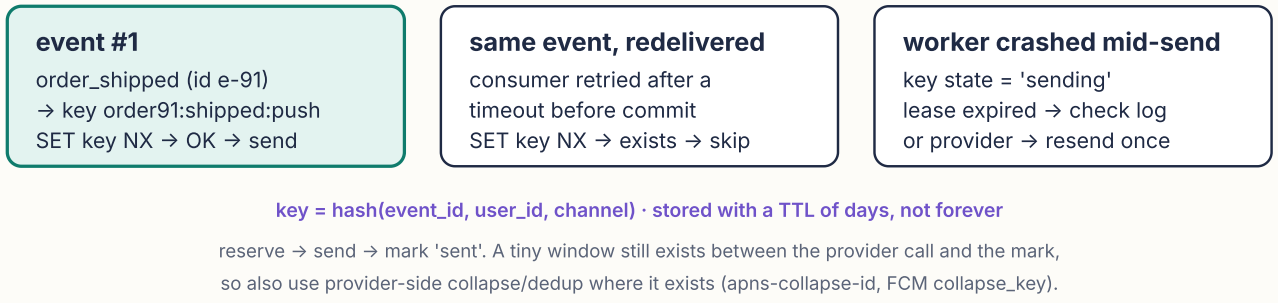


Figure 2. Three cases the key must handle. A plain "record then send" turns a crash into a lost message; "send then record" turns it into a duplicate. A short-lived 'sending' state with a lease narrows both windows.

The key covers *one logical notification*, so a genuinely new event (second shipment) gets a new key, while a redelivered one does not. Where the provider offers it, pass a collapse or dedup ID too: a second push with the same `apns-collapse-id` replaces the first on screen instead of showing twice.

WORKED EXAMPLE: ORDER SHIPPED VIA PUSH + EMAIL

The order service publishes `order_shipped {event_id: e-91, order: 91, user: 7}`. The notification service loads user 7's preferences: orders → push on, email on; it is 14:00 in their time zone, so no quiet-hours hold. It renders the push template ("Your order is on its way") and the email template (with tracking link) in their locale, and claims keys `order91:shipped:push` and `order91:shipped:email`. Both go to the transactional queue; the push leaves through FCM within a second or two, the email through the email provider. Ten minutes later Kafka redelivers e-91 after a consumer rebalance: both keys already exist, both sends are skipped, and the log shows *skipped: duplicate*.

TRADE-OFFS

- Central service: consistent rules and one place to add channels, but a critical shared dependency; give it priority isolation.
- Strict dedup window (days) costs storage; a short window risks duplicates from late redeliveries.
- Aggressive rate limits and digests reduce annoyance but can delay something the user wanted now.
- Multiple SMS/email providers improve availability but double the integration and template testing.

COMMON MISTAKES

- Checking preferences after rendering or after queueing, so a change made minutes ago is ignored.
- Quiet hours in server time instead of the user's time zone.
- One shared queue: the marketing blast delays every password-reset code.
- Never pruning dead device tokens or hard-bounced addresses, which hurts cost and email sender reputation.

6 Key terms

APNs / FCM: Apple and Google push gateways for device notifications

Preference store: per-user, per-category, per-channel opt-ins and quiet hours

Priority queue: separate lanes so critical messages never wait

Device token: per-install address for push; expires on uninstall

Idempotency key: event + user + channel; claimed once, duplicates skipped

Digest: many low-value notifications collapsed into one

7 Interview questions

Q1. What are the main parts of a notification system? BASIC

Events in, recipient and preference checks, per-channel templates, idempotency, priority queues, workers with retries and provider failover, per-user rate limits, and a send log.

Q2. How do you prevent duplicate notifications? BASIC

Give each notification an idempotency key from event, user and channel; claim it atomically before sending and skip keys already seen, while producers and consumers tolerate retried events.

Q3. A 5-million-user marketing campaign is delaying password-reset SMS. Fix it. SCENARIO

Split queues and worker pools by priority, throttle bulk sends, and reserve provider capacity for critical traffic.

Q4. Your main SMS provider starts timing out. What happens? SCENARIO

Retries with backoff absorb blips; a circuit breaker on rising error rates fails over to the second provider, and permanent errors are not retried.

Q5. How do you implement quiet hours? SCENARIO

Store the user's time zone and window; for non-critical messages inside it, schedule delivery for the window's end (or drop if stale), while security alerts bypass it.

Q6. Record the key before or after calling the provider? TRICKY

Before risks losing a message if you crash before sending; after risks a duplicate. Use a 'sending' state with a lease and reconcile on recovery, plus provider collapse IDs to hide any duplicate that slips through.

Q7. Why not let each service call FCM directly? TRICKY

Every service would re-implement preferences, quiet hours, templates, dedup and rate limits, and they would drift; users would get exactly the 3 a.m. duplicates you are trying to avoid.

Next: SD-Q-004 · Feed Generation backendroadmap.com